banana-brain: optimization pass (CPU and GPU)

Date: October 25, 2026 · **Sources**: `src/brain/network/*.cpp`, `src/brain/tools/compute.cpp`, the benchmark suite of this session (`suite`, `one`), `perf`, `nsys`

> **How to read this document**
>
> 1. What was measured and how.
> 2. The hot spots found (CPU and GPU).
> 3. The changes, each with its cause and its effect.
> 4. The results before / after, and what they do not prove.
>
> **Limits**: the machine was heavily loaded during the pass (load average 13 to 19, other programs of the user running): the "before" is one run, the "after" the best of 3 runs; small differences (< x1.5) are within the noise, the big ones are not. The kernel times of `nsys` do not depend on the load. GPU: RTX 3050 Laptop; build `Optimized` (`-O3 -ffast-math`).

Contents

1. Summary
2. Scenario
3. Hot spots
4. Changes
5. Results
6. What is left

1. Summary

Point	Conclusion
Results	the same: 149 tests pass; checksums identical except one CPU case (-ffast-math rounding, 5e-8 relative) and the GPU of 2000 hidden neurons (atomic additions in any order)
CPU, brain	500 hidden neurons: 71 -> 22 ms per round (plain), 2000: 1076 -> 368 ms; execution 4.2 -> 1.9 ms per sample
CPU, layered	{1024,1024,10}: 115 -> 21 ms per round (no copy of the weights per sample any more)
GPU, small networks	brain 13 hidden: 29.1 -> 2.8 ms per round (no allocation per call, no new copy of the execution copy per update); layered {16,16,1}: 27.5 -> 11 ms
GPU, kernels	backpropagation step 605 -> 239 us at 2000 hidden neurons, forward step 19 -> 5.3 us at 500 (one warp per neuron)
Left	the training options (Adam, mini-batch) still run on the host on a GPU network; the GPU round trips per sample dominate small networks

2. Scenario

`suite` (one binary linked on the library): for each case, rounds of training then the execution of every sample, with a checksum of the outputs (it must not change). Brain networks of 13, 60, 500 and 2000 hidden neurons (plain gradient and Adam + mini-batch, 4 or 81 samples), layered networks {16,16,1}, {256,256,10}, {1024,1024,10} (64 / 16 samples), each in RAM and on the GPU. Profiles: `perf record` (CPU), `nsys` (CUDA kernels and API).

3. Hot spots

Case	Before: where the time went	Cause
layered, RAM	55 % memmove / memset	`learn_` copied every weight out of the memory and back for each sample

Case	Before: where the time went	Cause
brain execution, RAM	71 % `BrainNetwork::compute_`	every slot read, also the free ones of the execution copy (about half)
brain training, RAM	33 % heap of the sort	the init sorted the candidates of every neuron with a partial heap sort
brain update	`std::function` per slot, clone search	two `std::function` calls per weight, the clone of each link searched in the whole cap (n x cap^2)
GPU, every call	waiting in the driver	`cuMemAlloc` / `cuMemFree` of the scratch buffers at every call, a synchronize before a blocking copy
GPU, brain update	allocation of the execution copy	`sync_` allocated the device memory of the execution copy at every update
GPU kernels, large cap	uncoalesced reads, clone loop	one thread per neuron reads slots `cap` links apart; `back_update` searched the clone of each link (cap^2)

4. Changes

1. **Layered network learns in place.** `LayerNetwork::learn_` works on the host memory directly (a GPU memory that falls back is still fetched / stored); `forward_` sums in a local variable; the error buffers swapped instead of allocated per layer.
2. **Execution copy stops at the first free slot.** `BrainNetwork::compute_`: the inward links are packed first, a weight of 0 is a free slot (a real weight is never 0).
3. **Nearest lists with `nth_element`.** the `2 cap + 8` nearest candidates found with `nth_element` then sorted (same order: the id breaks the ties).
4. **Update without `std::function`.** `change_` (Adam / momentum / step limit) is a member function called directly.
5. **Clones known, not searched.** `_clones[slot]` = the slot of the clone of an inward link: built once (init / load), kept up to date by `relink_` for each new link (a removal needs nothing), used by the update, `weightInit` and the GPU (`_cloneMemory`).
6. **No new device memory per update.** `sync_` keeps the memory of the execution copy when its size does not change.
7. **GPU scratch buffers kept.** per thread and per context (`Scratch`), grown when too small, checked against their context: no `cuMemAlloc` / `cuMemFree` per call; the synchronize before a blocking copy removed.
8. **GPU warp kernels.** `step_warp` and `back_step_warp` (cap >= 32): one warp per neuron, the lanes read consecutive slots, the sum reduced in the warp; `back_update` writes the clone from `_cloneMemory`.

5. Results

Case	Train before (ms/ round)	Train after	Gain	Exec before (us/ sample)	Exec after	Checksum
brain 13 hidden ram plain	0.181	0.063	x2.9	8.7	5.2	same
brain 13 hidden ram adam	0.121	0.054	x2.3	12.4	5.0	same
brain 60 hidden ram adam (81 s.)	26.174	8.922	x2.9	133.0	44.6	839.813299 -> 839.813257
brain 500 hidden ram plain	71.319	22.449	x3.2	4172.8	1927.0	same
brain 500 hidden ram adam	50.596	21.873	x2.3	3972.6	1864.8	same
brain 2000 hidden ram plain	1076.369	368.173	x2.9	32147.7	16633.1	same
brain 13 hidden gpu plain	29.100	2.803	x10.4	108.4	85.8	same
brain 13 hidden gpu adam	0.875	0.538	x1.6	158.8	80.5	same
brain 500 hidden gpu plain	22.348	5.000	x4.5	695.5	217.0	same
brain 500 hidden gpu adam	42.709	26.198	x1.6	547.6	232.1	same
brain 2000 hidden gpu plain	352.123	573.853	x0.6	9815.4	9643.0	0.000008 -> 0.002652
layer {16,16,1} ram	0.171	0.101	x1.7	1.0	0.9	same
layer {256,256,10} ram	13.001	4.714	x2.8	40.9	25.1	same
layer {1024,1024,10} ram	115.042	21.133	x5.4	1062.0	405.7	same
layer {16,16,1} gpu	27.541	11.003	x2.5	115.2	36.5	same
layer {256,256,10} gpu	28.828	20.958	x1.4	153.0	95.8	same
layer {1024,1024,10} gpu	34.734	29.593	x1.2	680.6	550.5	same

GPU kernel (nsys)	Before	After
backpropagation step, 2000 hidden (cap 134)	605 us	239 us (warp)
forward step, 2000 hidden	229 us	234 us (warp: no gain at this size)
forward step, 500 hidden (cap 67)	19.1 us	5.3 us (warp)

> **Key takeaway**
> The gains above x2 are real (allocations, copies, clone searches removed); the small differences of the small cases are the noise of the loaded machine (the before is a single run).

Checksums: one CPU case moved by 5e-8 relative (the Adam step moved from a lambda to a member function: under `-ffast-math` the compiler rounds it differently; checked by putting the lambda back); the GPU case of 2000 hidden neurons changes between runs (the order of the atomic additions of the warp kernel), as GPU results already differed from the host within a tolerance (the GPU tests compare at 1e-4).

6. What is left

Change	Gain	Effort
Adam / momentum / mini-batch on the GPU (kernels of the update)	large networks on the GPU with the training options (today read back to the host)	medium
Keep the states of the forward pass on the device for the backpropagation	one upload of every state per sample less	low
Batch of samples in one GPU call	small networks on the GPU (the round trips per sample dominate)	high

Change	Gain	Effort
Execution copy rebuilt only when read	the host copy of the reduced memories per update	low